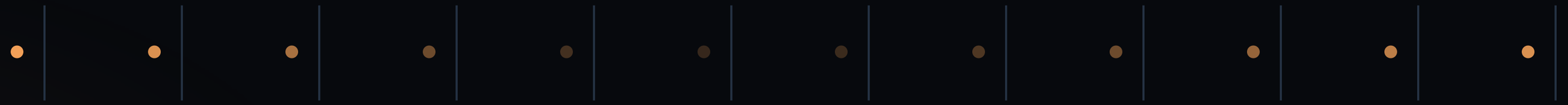
ONSITE PRESENTATION

Waking up on time

Periodic notification fences for indirect displays, and why late-stage reprojection depends on them

Shantanu Gupta



ABOUT ME

Working where hardware meets software

2023 – now
Apple · Platform Diagnostics, Vision Products Group

2022 – 2023
Intel · Microsoft Co-Engineering

2019 – 2022
Microsoft · dxgkrnl, Silicon Graphics & Media

2015 – 2019
Drexel University · B.Sc. Computer Science

C · C++ · Python · WinDbg · WPA/xperf · GPUView · PIX · lldb/gdb · IDA Pro

INTERESTS

HW/SW interface · Firmware & OS · VR & novel displays · Domain-specific architectures · Parallel computation · Aviation & automotive

INTERNSHIPS

Microsoft	Summer 2018
Markforged	Spring 2018
Microsoft	Summer 2017
Sourcegraph	Fall 2016
Palo Alto Networks	Summer 2016

PERSONAL PROJECTS

cLK	Homebrew Linux bootloader for Windows Mobile phones, on Little Kernel	2012
htc-linux	Kernel and Android tree for CyanogenMod on the HTC HD2	2012
Windows for Vision Pro	Virtual displays via UIKit, CoreGraphics and IddCx	2024
μ-2σ	Best Halite II bot at Two Sigma	2018

THE PROBLEM

In a headset, latency feels like nausea

COMPOSITOR

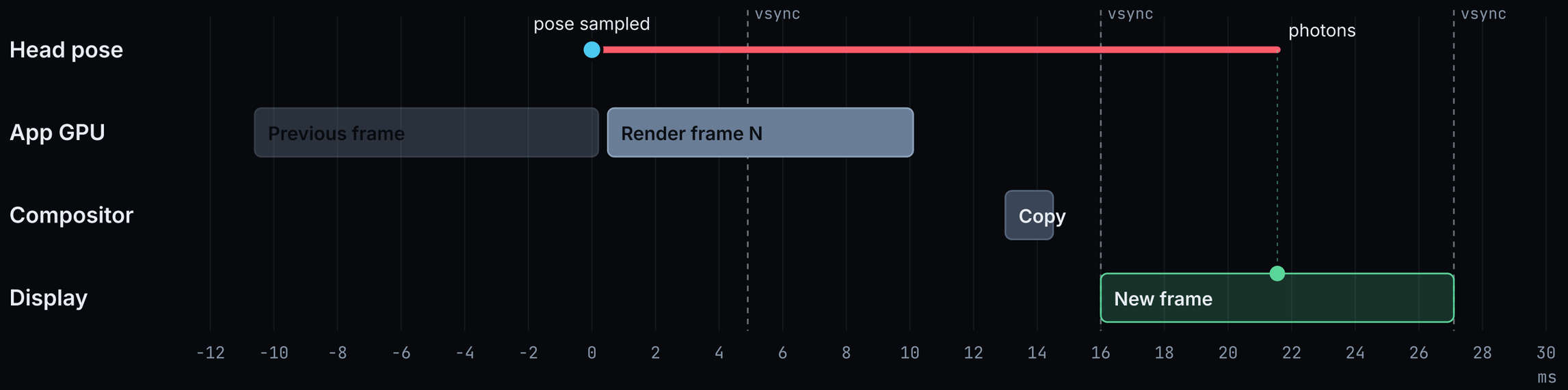
Plain compose | Late-stage reprojection

APP FRAME

On time | Misses vsync

POSE → MID-SCREEN PHOTONS

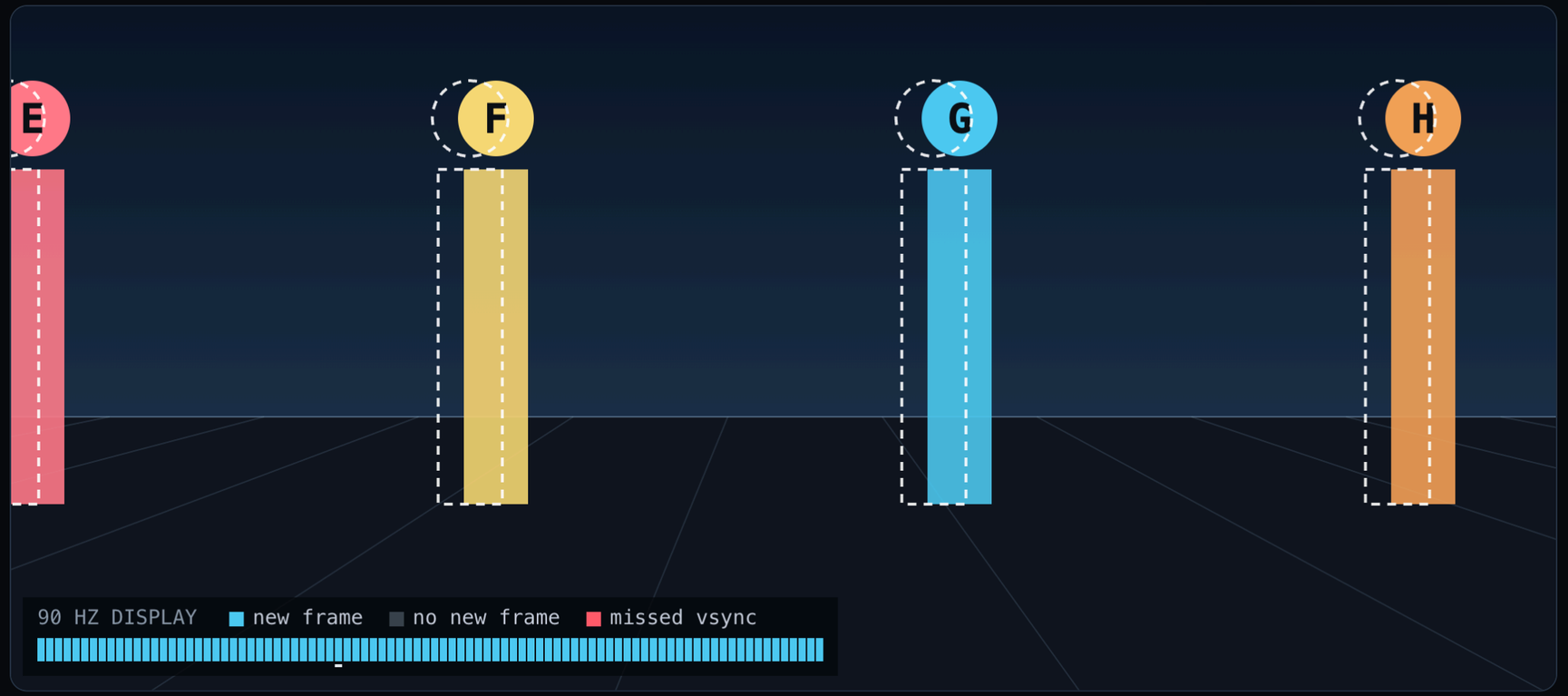
21.6 ms



Illustrative 90 Hz pipeline. Roughly 20 ms motion-to-photon is a commonly cited comfort target.

DEMO

Warp the last frame to the newest head pose



LATE-STAGE REPROJECTION

Off | On

RENDER PIPELINE LATENCY 30 MS

APP FRAME RATE

90 | 45 | 20

FRAMES THAT MISS VSYNC 0%

HEAD MOTION

Auto sweep | Follow mouse

WHERE IT SHOULD BE

Show | Hide

AVG. IMAGE ERROR

1.8°

HOW IT WORKS

The warp is cheap. Starting it on time is hard.

1. App renders a frame using the head pose predicted at render start
2. Just before scanout, sample the newest head pose
3. Warp the finished image by the rotation since the render pose
4. Scan it out. Repeat every refresh, even if the app missed its frame

$$H = K \cdot R_{\Delta} \cdot K^{-1}$$

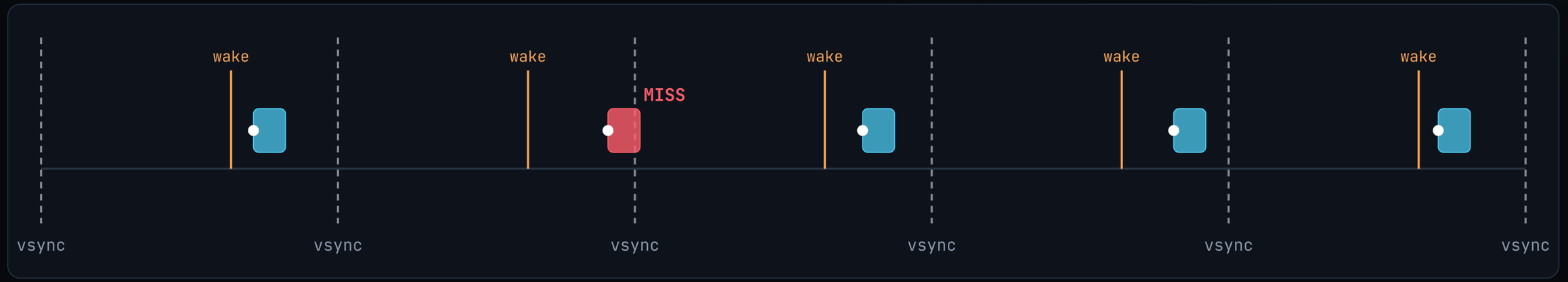
- K projection intrinsics
- R_{Δ} head rotation between render pose and latest pose

- One full-screen resample per eye: small, fixed cost
- Translation and parallax need depth
- Uncovered edges need a guard band

> Hard constraint: the warp must **start late enough** to use a fresh pose and **finish before vsync**, every frame.

THE SCHEDULING PROBLEM

When should the LSR thread wake up?



WAKE-UP SOURCE

Coarse timer | Vsync-locked fence | Custom

WAKE OFFSET BEFORE VSYNC 4.0 MS

LSR COST 1.2 MS

WAKE-UP JITTER (MEAN) 1.20 MS

MISSED VSYNCS

9.71%

POSE → MID-SCREEN PHOTONS

8.4 ms

> Wake too early → stale pose. Wake too late → missed vsync. **Jitter** decides how close you can cut it.

THE PRIMITIVE

A fence signaled every refresh, offset from vsync

- Signaled by the OS every refresh, a fixed offset before vblank. The value counts up by one each time
- Waitable like any GPU fence by CPU threads and GPU queues. No polling, no sleep()
- On a GPU, the display driver drives it from its vblank interrupt

SAME IDEA, THREE LAYERS

WINRT	`DisplayDevice.CreatePeriodicFence(target, offsetFromVBlank)`
D3DKMT	`D3DDDI_PERIODIC_MONITORED_FENCE`
KMD DDI	`DxgkDdiCreatePeriodicFrameNotification` offset in 100 ns, at most one refresh

> The catch: **indirect displays** have no display engine on the GPU and no vblank interrupt, so there is nothing to drive the fence.

The Indirect Display driver model (IddCx) is how Windows creates virtual and remote monitors: USB docks, wireless and virtual displays. Its main partners were DisplayLink and Intel.

THE PLUMBING

How a vsync reaches dxgkrnl

GPU display driver

full WDDM kernel-mode driver

HARDWARE

Display controller enters vblank

↓ IRQ

KMD ISR

`DxgkCbNotifyInterrupt(DXGK_INTERRUPT_CRTC_VSYNC)`

from DxgkDdiInterruptRoutine

↓ DPC

DXGKRNL

Flip retires; present statistics advance

↓

PERIODIC FENCE

`DxgkDdiCreatePeriodicFrameNotification`

Driver raises DXGK_INTERRUPT_PERIODIC_MONITORED_FENCE_SIGNALED at vsync − Time

Display-only driver with VSync

KMDOD that reports interrupts

HARDWARE

Scan-out hardware with a vblank interrupt

↓ IRQ

KMDOD ISR

`DXGK_INTERRUPT_DISPLAYONLY_VSYNC`

Requires DxgkDdiControlInterrupt + DxgkDdiGetScanLine

↓ DPC

DXGKRNL

`DxgkDdiPresentDisplayOnly`

Async completion via DXGK_INTERRUPT_DISPLAYONLY_PRESENT_PROGRESS

↓

PERIODIC FENCE

No DDI: KMDDOD_INITIALIZATION_DATA has no DxgkDdiCreatePeriodicFrameNotification

Display-only without VSync · IddCx

remote and virtual monitors

REMOTE DEVICE

The panel's TCON owns the real vblank, out of the host's reach

↓ no IRQ

DRIVER

`vSyncFreq = D3DKMDT_FREQUENCY_NOTSPECIFIED`

No DxgkDdiControlInterrupt, no DxgkDdiGetScanLine

↓

DXGKRNL

Simulates VSync interrupts and scan lines from the current mode and the time of the last simulated VSync

↓

PERIODIC FENCE

The emulation thread arms a high-res timer at vblank − offset; the CPU signals the fence (WARP path)

THE CONSUMER

One frame of the VR compositor

SETUP · C++/WINRT + D3D12

```
auto interop = device.as<IDisplayDeviceInterop>();
DisplaySource   source = device.CreateScanoutSource(target);
DisplayTaskPool pool   = device.CreateTaskPool();

for (int i = 0; i < kBuffers; ++i) {
    primary[i] = device.CreatePrimary(target, primaryDesc);
    HANDLE h;
    interop->CreateSharedHandle(primary[i].as<IInspectable>().get(),
                                nullptr, GENERIC_ALL, nullptr, &h);
    d3d->OpenSharedHandle(h, IID_PPV_ARGS(rt[i].put()));
    scanout[i] = device.CreateSimpleScanout(source, primary[i], 0,
                                            /*syncInterval*/ 1);
}

// D3D-created fences, wrapped as DisplayFences: renderDone, flipTo, flipAway
// (CreateFence(SHARED) -> CreateSharedHandle -> interop->OpenSharedHandle)
DisplayFence renderDoneDF = WrapD3DFence(renderDone);
DisplayFence flipToDF     = WrapD3DFence(flipTo);
DisplayFence flipAwayDF   = WrapD3DFence(flipAway);

// Display-created periodic fence, exposed to D3D
// (interop->CreateSharedHandle -> d3d->OpenSharedHandle)
DisplayFence tick = device.CreatePeriodicFence(target, offsetFromVblank);
com_ptr<ID3D12Fence> tickD3D = OpenInD3D(tick);
```

PER FRAME · BUFFER I, TARGET REFRESH K

```
queue->Wait(flipAway.get(), lastIdUsing[i]);   // buffer i is off-screen
queue->Wait(tickD3D.get(), k);                 // start at vblank-offset, display-timed
queue->ExecuteCommandLists(1, &warpInto_i);    // shader reads late-latched pose
queue->Signal(renderDone.get(), ++v);

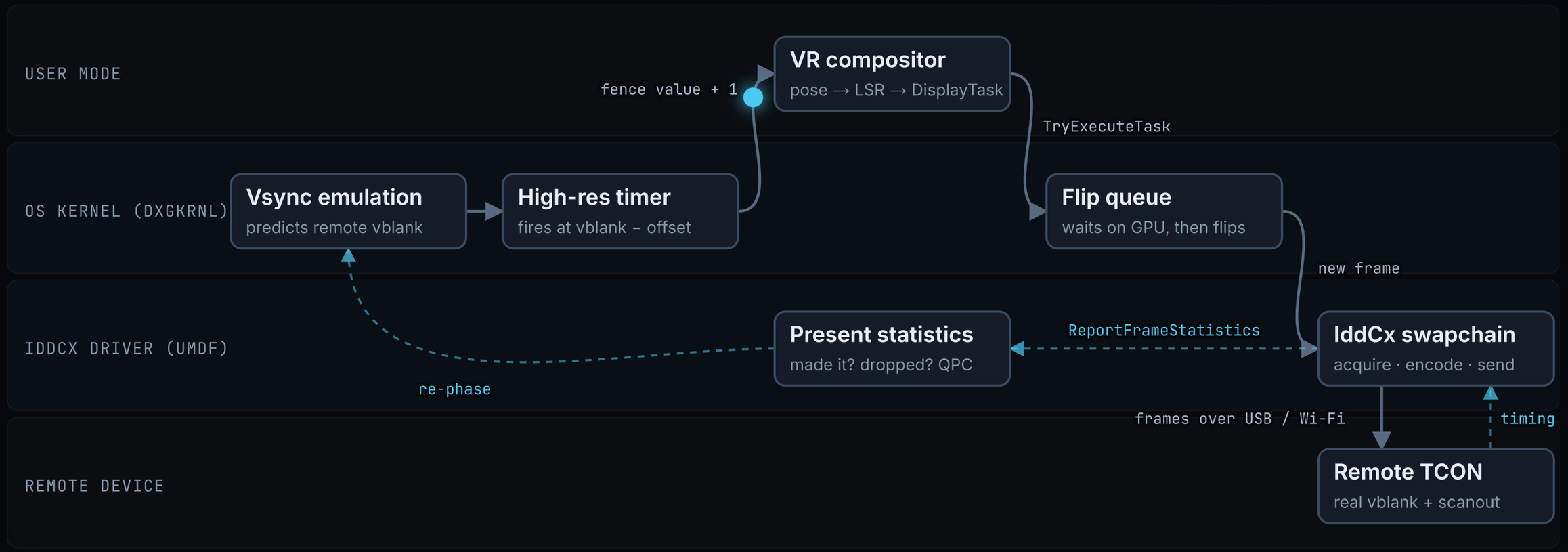
DisplayTask task = pool.CreateTask();
task.SetScanout(scanout[i]);
task.SetWait(renderDoneDF, v);                 // flip gated on GPU completion
task.SetSignal(DisplayTaskSignalKind::OnPresentFlipTo,   flipToDF);
task.SetSignal(DisplayTaskSignalKind::OnPresentFlipAway, flipAwayDF);
DisplayTaskResult r = pool.TryExecuteTask(task);   // returns immediately
lastIdUsing[i] = r.PresentId();
targetRefreshFor[r.PresentId()] = k + 1;
```

FENCE	DIRECTION	SIGNALED
tick	display → GPU	every refresh at vblank − offset: the warp waits for value k
renderDone	GPU → display	when the warp finishes: the flip waits for it
flipTo	display → app	when the present reaches the screen
flipAway	display → GPU	when buffer i leaves the screen: the next warp into it waits

The warp waits for the display (tick), and the flip waits for the GPU (renderDone). TryExecuteTask only queues the flip and returns.

WHAT I BUILT

DisplayTask on an indirect display



UNDER THE HOOD

Frames go out, timing comes back

IDDCX DRIVER · USER MODE

```
// One thread per swapchain, as in the IddCx sample driver
IddCxSwapChainSetDevice(swapChain, &setDevice);
for (;;) {
  IDARG_OUT_RELEASEANDACQUIREBUFFER buf = {};
  HRESULT hr = IddCxSwapChainReleaseAndAcquireBuffer(swapChain, &buf);
  if (hr == E_PENDING) { WaitForNewFrame(); continue; }

  // buf.MetaData.pSurface: frame to encode
  // buf.MetaData.PresentDisplayQPCTime: when it should be on screen
  bool sent = EncodeAndSend(buf.MetaData.pSurface);
  IddCxSwapChainFinishedProcessingFrame(swapChain);

  stats.FrameStatus = sent ? IDDCX_FRAME_STATUS_COMPLETED
                           : IDDCX_FRAME_STATUS_DROPPED;
  IddCxSwapChainReportFrameStatistics(swapChain, &args);
}
```

DXGKRNL VSYNC EMULATION · PSEUDOCODE

```
timer = ExAllocateTimer(OnFenceTimer, ctx,
                        EX_TIMER_HIGH_RESOLUTION);

OnFlip():                                  // vsync emulation thread
  dueQpc = PredictVblankQpc(stats) - offsetQpc;  // QPC ticks
  nowQpc = KeQueryPerformanceCounter(&freq);
  delay  = (dueQpc - nowQpc) * 10'000'000 / freq;  // ⟶ 100 ns
  ExSetTimer(timer, -delay, 0, NULL);   // high-res: relative only

OnFenceTimer():
  fence.value += 1;
  SignalPeriodicFence(fence);      // compositor wakes up

OnModeChange():
  fence.value = MAX;               // release waiters exactly once
  SignalPeriodicFence(fence);
```

PresentDisplayQPCTime

The OS tells the driver when each frame should reach the remote screen.

COMPLETED / DROPPED

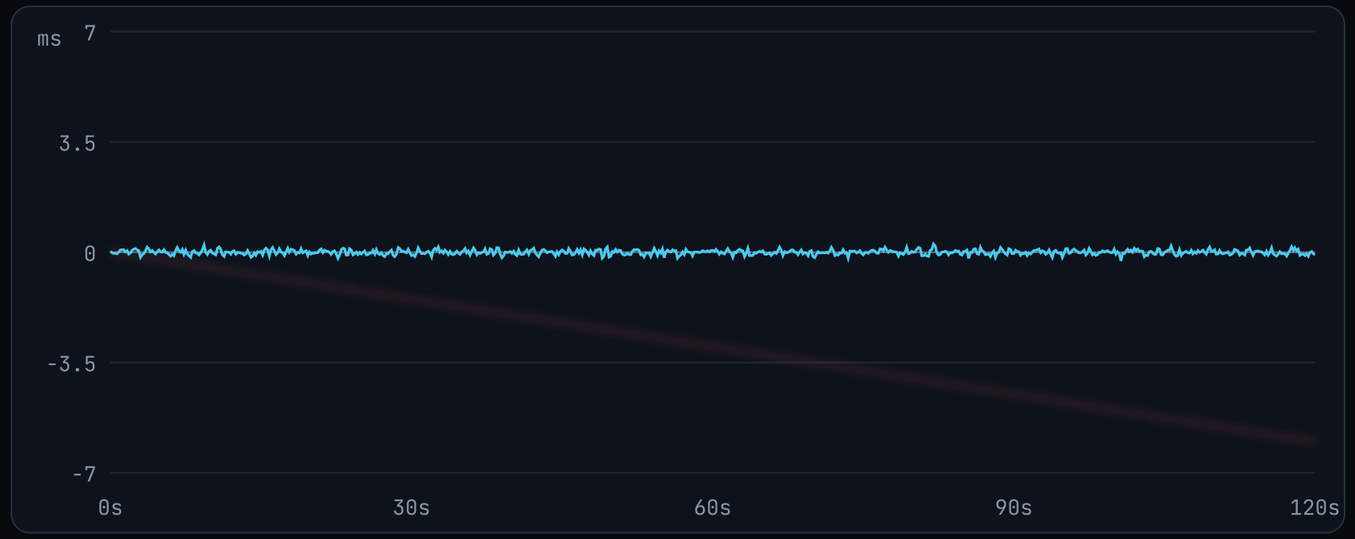
Per-frame status and send timestamps flow back to the OS.

High-res timers

Relative due time in 100 ns. Up to ~1 ms late, never early. Re-armed every flip.

STAYING IN SYNC

Staying in phase with a panel you can't see



CLOCK MISMATCH **50 PPM** FEEDBACK TIMESTAMP NOISE **0.30 MS**

FEEDBACK LOOP

Free-running | **Re-phase every flip**

FREE-RUNNING AFTER 2 MIN
6.0 ms

LOCKED, P99 |ERROR|
199 µs

OUT →
`IDDCX_METADATA.PresentDisplayQPCTime`
when the OS wants this frame on the remote screen

← BACK
`IddCxSwapChainReportFrameStatistics`
`FrameStatus` COMPLETED / DROPPED, `FrameAcquireQpcTime`, `SendStart/Stop/CompleteQpcTime`

KEPT
`D3DKMT_PRESENT_STATS`
`PresentRefreshCount` vs `PresentCount`: a jump > 1 is a missed vblank. Surfaced as `IDXGIOutput::GetFrameStatistics`

↻ LOOP
re-phase every flip
the emulated vblank, and the periodic fence riding on it, follow the feedback

EDGE CASES

Edge cases to consider

The remote panel drifts

The real vblank fires on another device, on its own crystal.

The driver reports whether each flip made it (IDDCX_FRAME_STATUS), and the present statistics show missed vblanks (PresentRefreshCount vs. PresentCount). That feedback keeps the emulated vsync in phase with the remote TCON.

The mode changes mid-wait

Every mode change recreates the primaries and the swapchain.

Pending periodic fences are set to their max value and signaled exactly once, so no waiter hangs. On its next present the compositor sees the swapchain is gone, recreates its primaries and gets the new swapchain from IddCx.

A second fence is requested

One timer per device keeps the kernel thread simple.

The fence lives inside the DisplayTask object, which owns all contact with the vsync emulation thread. That thread arms its own high-res timer only while a periodic fence exists, so a display-only device gets at most one. Fine: the AR/VR/MR compositor is its only client.

The deadline and the timer use different clocks

QPC ticks above, 100 ns timer units below.

The compositor, IddCx and DisplayTask keep time in QueryPerformanceCounter ticks. The high-resolution timer wants a relative delay in 100 ns units. Every arm converts QPC deadline → 100 ns delay (KeQueryInterruptTimePrecise returns both clocks from one read). Rounding or a stale "now" lands directly in the jitter.

RECAP

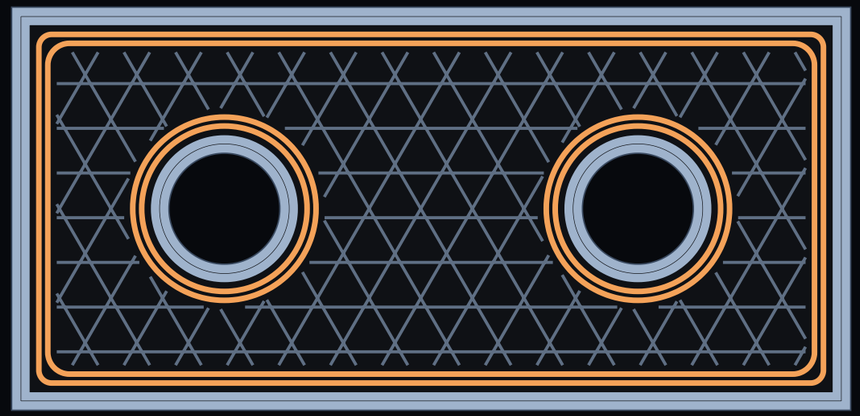
- LSR makes head motion feel instant, if it runs on time
- Periodic fences turn "on time" into an OS primitive
- On indirect displays, the fence rides an emulated vsync kept in phase by present statistics

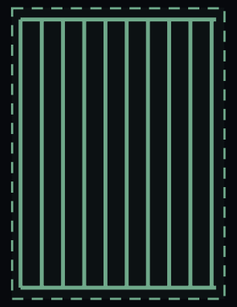
Questions?

Shantanu Gupta

BONUS · MARKFORGED SLICER

What a slicer does with one layer



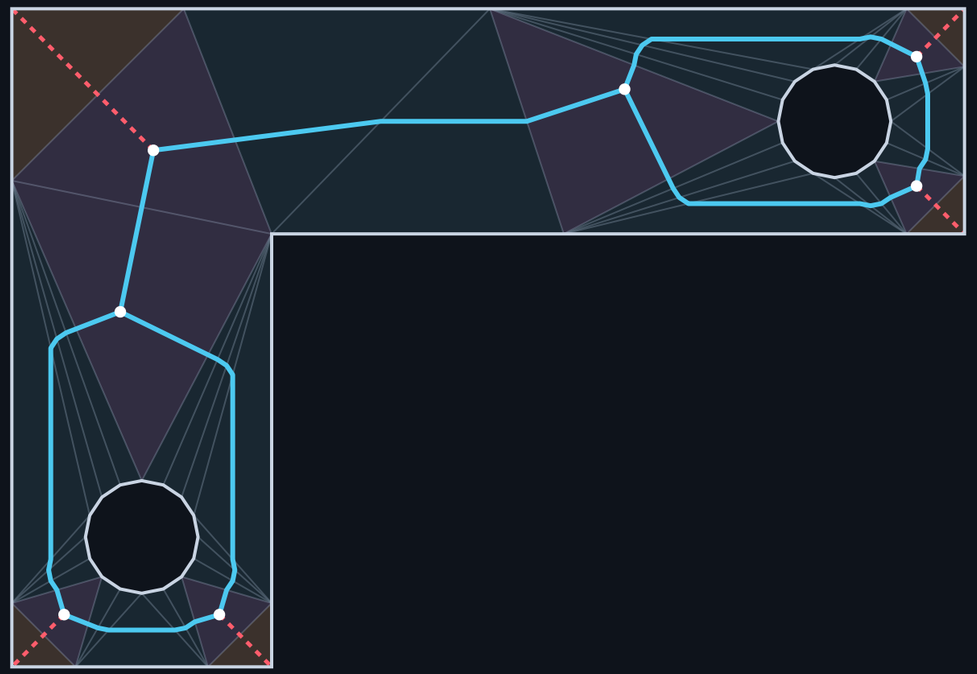


TOOLPATH	GENERATED BY	MAIN CONSTRAINT
Walls	inward offsets of the outline	bead width, smallest feature
Solid skin	dense lines on top and bottom layers	must close the surface
Infill	a lattice clipped to the region	needs room for the pattern
Support	sparse pattern under overhangs	overhang angle, easy removal
Carbon fibre	rings or routed strands	minimum length, tightest bend

Continuous carbon fibre stiffens the part: **bending, torsional and shear rigidity** well beyond plastic alone. But a strand has a **minimum length** and a **tightest bend radius**.

BONUS · MEDIAL AXIS

Approximating the medial axis with a triangulation



Step 6 / 6

SHAPE

C-channel | **Bracket** | Tapered rib | Cross

0 Layer polygon

1 Constrained Delaunay triangulation

2 §2.1 Fix obtuse junction triangles

3 §2.2 Fix convex corners

4 §2.3 Classify triangles

5 Build the axis

6 Drop the segments that reach the wall

Our change for the slicer: the last segment of every branch runs into a convex corner on the wall. A fibre laid along it would come out through the surface, so it is removed and the axis stops at the junction.

A · 40 | B · 5 | C · 7 | dropped · 5

38 vertices · +2 Steiner (§2.1) · +10 Steiner (§2.2) · 52 triangles

After Smogavec & Žalik, "A fast algorithm for constructing approximate medial axis of polygons, using Steiner points", Advances in Engineering Software 52 (2012).

BONUS · FROM AXIS TO TOOLPATH

Two jobs for the same skeleton

Carbon fibre: break the axis, then rebuild it

C-channel | Bracket



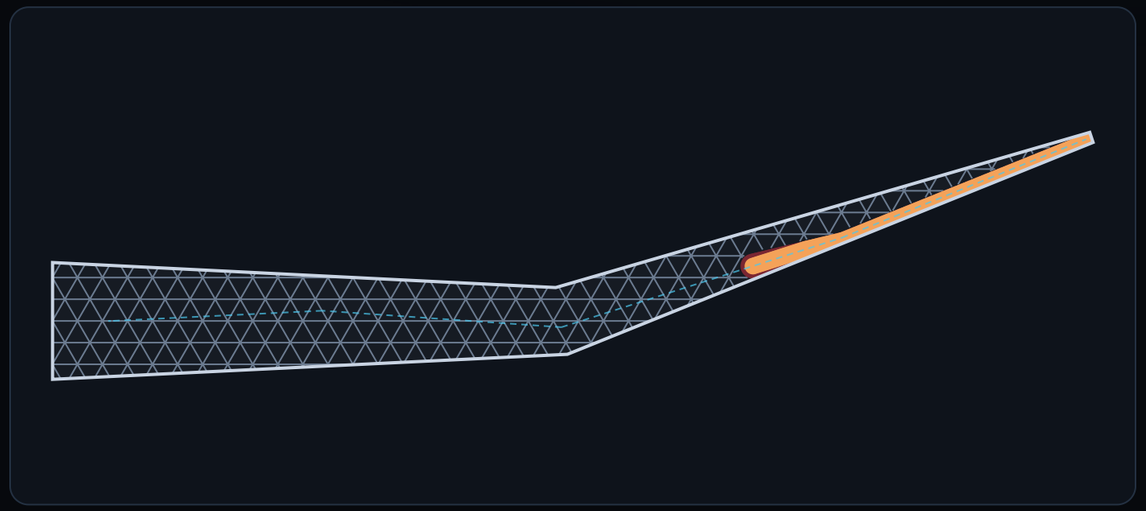
MINIMUM STRAND LENGTH **30 MM**

TIGHTEST BEND RADIUS **4 MM**

fibre strand · 1 — too short · 0 — bend too tight · 0 — 192 mm laid

Thin regions: one bead along the axis

Infill only | + axis bead



INFILL PATTERN NEEDS AT LEAST **3.0 MM**

infill pattern — slight gap left — plastic bead, no fibre (≤ 2 mm wide)

Region left inside the walls of a rib: 14 mm wide at the root, 1.2 mm at the tip.

References

- **Windows.Devices.Display.Core**: DisplayDevice.CreatePeriodicFence, DisplayTask (SetScanout, SetWait, SetSignal), DisplayTaskPool.TryExecuteTask, DisplayTaskResult.PresentId
- **Building a custom compositor for head-mounted and specialized monitors** (WDK docs)
- **IDisplayDeviceInterop**: CreateSharedHandle / OpenSharedHandle, share primaries and fences with D3D12
- **Indirect Display Driver model overview** and the IndirectDisplay sample, Windows-driver-samples
- **IddCxSwapChainReleaseAndAcquireBuffer**, IDDCX_METADATA, IddCxSwapChainReportFrameStatistics, IDDCX_FRAME_STATISTICS
- **D3DDDI_PERIODIC_MONITORED_FENCE**, DXGKARG_CREATEPERIODICFRAMENOTIFICATION
- **High-resolution timers**, Timer Accuracy, ExAllocateTimer, ExSetTimer, KeQueryInterruptTimePrecise
- **Saving Energy with VSync Control** (display-only VSync), KMDDOD_INITIALIZATION_DATA, DXGK_INTERRUPT_TYPE, DXGKARGCB_NOTIFY_INTERRUPT_DATA
- **D3DKMT_PRESENT_STATS**, DXGI_FRAME_STATISTICS, IDXGIOutput::GetFrameStatistics

---

MEDIAL AXIS

Smogavec & Žalik, **"A fast algorithm for constructing approximate medial axis of polygons, using Steiner points"**, Advances in Engineering Software 52 (2012) 1–9